12/01/26

Q1. Create & pre-processes a student dataset using numpy & pandas. Calculate the no of male & female in gender column of dataframe which contains

Columns - roll no., marks, attendance, gender

→ import numpy as np
import pandas as pd
n = np.array([[1,2,3,4,5,6,7,8,9,10],
[78,90,67,89,87,77,90,80,99,78],
[87,98,77,67,89,78,98,66,78,66]
['M','F','F','F','M','F','M','F','M','F']]

b = np.transpose(n)
df = pd.DataFrame(b, columns = ["RollNo","Marks","Attendance","Gender"]

df1 = df['Gender'].sum().count("F")
df2 = df['Gender'].sum().count("M")
df
print("Females are:", df1)
print("Males are:", df2)

OUTPUT :-

	Roll no	Marks	Attendance	Gender
0	1	78	87	M
1	2	90	98	F
2	3	67	77	F
3	4	89	67	F
4	5	87	89	M
5	6	77	78	F
6	7	90	98	M
7	8	80	66	F
8	9	99	78	M
9	10	78	66	F

Females are :- 6

Males are : 4

Q2. Using matplotlib plot the following graph

i) bar graph for gender distribution

Y = students X = gender

ii) Histogram for marks

the graphs should have proper title, labels.

i) gender = ["F", "M"]

std = [48, 56]

plt.bar(gender, std)

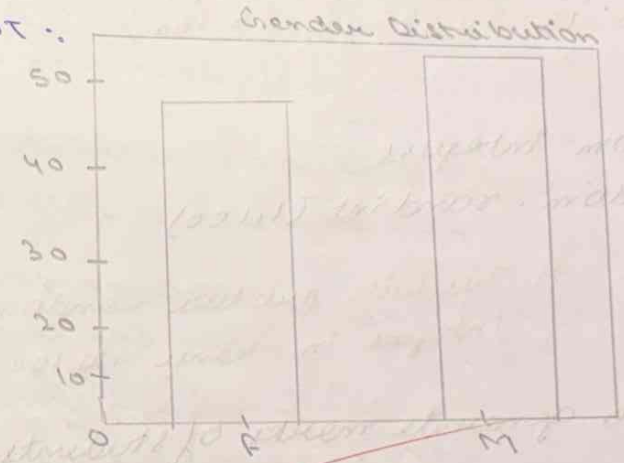
plt.xlabel("gender")

plt.ylabel("students")

plt.title("Gender Distribution")

plt.show()

OUTPUT :-



ii) marks = [67, 99, 89, 78, 66]

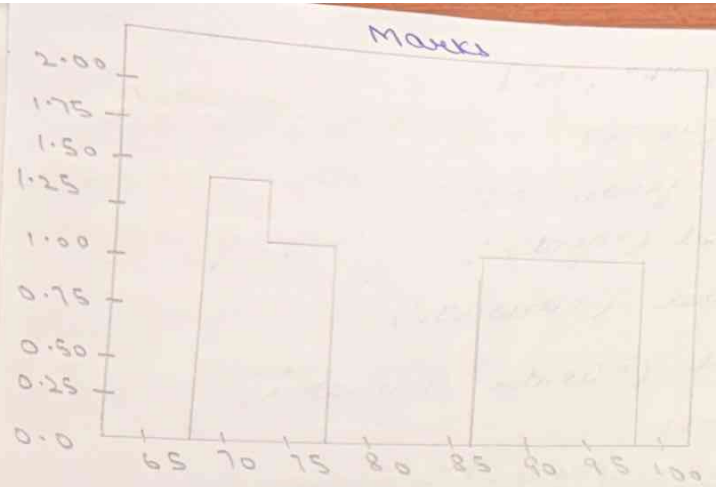
plt.hist(marks, bins=5)

plt.xlabel("Marks")

plt.title("Marks")

plt.show()

OUTPUT :-



→ for random integers

np. random. randint (1,100)

This will produce random integers in range 1 to 100.

Q3. WAPC to generate marks of students where marks are randomly generating & store the marks in list.

⇒ l = [ ]

for i in range (30):

a = np. random. randint (0,100)

l. append (a)

print ("marks :", l)

OUTPUT :

marks : [81, 85, 11, 80, 82, 17, 11, 8, 20, 91, 95, 72, 79, 49, 57, 64, 0, 2, 40, 91, 39, 27, 63, 54, 42, 11, 4, 68, 7, 38]

note : random values are generated here. It will give different output everytime.

for random values from normal distribution :

a = np. random. randn ( )

a

Q4. generate 5 random values using randn ( ) & plot a graph for it.

⇒ a = np. random. randn (5)

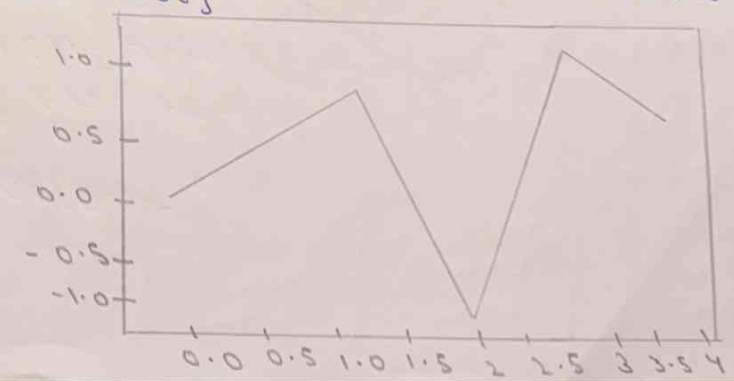
print (a)

plt. plot (a)

plt. show ( )

OUTPUT :

[0.03418431, 0.65292473, -1.28375625, 0.89336344, 0.854000]



• BFS CODE :-

```
def bfs (st, goal, graph):
    vis = set ()
    queue = [st]
    path = [ ]

    vis. add (st)

    while queue:
        node = queue. pop(0)
        path. append (node)

        if node == goal:
            return path

        for neighbour in graph [node]:
            if neighbour not in vis:
                vis. add [neighbour]
                queue. append (neighbour)

return path
```

19/01/26

Q5. WAPC to open a CSV file as dataframe & named (medical- records) & perform

(i) head
(ii) tail
(iii) describe
(iv) Info
(v) shape/size
(vi) find null value
(vii) find no of null values.

→ df = pd. read_CSV ("medical_records. csv")
df

Output

	record_id	patient_id	diagnosis	bp	sugarlevel	visit cost
0	R101	P013	Hypertension	120/80	140.0	2000
1	R102	P015	Diabetes	120/90	NaN	2500
2	R103	P038	Asthma	130/85	NaN	1500
118	R219	P011	Hypertension	120/85	180.0	2500
119	R220	P013	Hypertension	High	180.0	1500

(i) df. head (5)

Output.

0	R101	P013	Hypertension	120/80	140.0	2000
1	R102	P015	Hypertension	120/80	NaN	2500
2	R103	P038	Hypertension	High	140.0	3000

ii) df. tail (5)

Output.

115	R216	P081	Asthma	High	110.0	2500
116	R217	P033	Diabetes	High	NaN	1500
117	R218	P063	Hypertension	140/90	NaN	2000

(iii) df. describe()

Output

	Sugar level	Visit - cost
Count	8.50000000	12.0000000
Mean	141.058824	2292.50000000

(iv) df. info()

Output

#	Column	Non - null count	D-type
0	record - id	120	Object
1	patient_id	120	Object.

(v) df. size ← no of elements.

Output

→ 720.

(vi) df. shape

Output

→ (120,6)

(vii) df. isna()

Output

#	record - id	patient_id	diagnosis	bp	Sugar level	visit
0	False	False	False	False	False	F
1	False	False	False	False	True	F

(viii) df. isna().sum()

Output

record - id	0
patient_id	0
diagnosis	0
bp	0
sugar level	35
visit - count	0.

(viii) Replace all the NaN values in sugar - level column with 0.

→ df["sugar-level"] = df["sugar-level"].fillna(0)
df.

Output:

#	record - id	patient_id	diagnosis	bp	sugar level	Visit
0	R101	P013	Hypertension	120/80	140.0	2000
1	R102	P015	Diabetes	120/80	0	250
2	R103	P098	Hypertension	High	140.0	3000

Q6. WAPC to read a CSV file named (patient-details.csv) perform following

→ df1 = pd.read_csv("patient_details.csv")
df1

Output :-

*	patient - id	name	age	gender	city	Adm - date	heig
0	P001	NAN	25.0	F	Mumbai	15/01/[illegible]	1[illegible]
1	P002	Rahul Sharma	60.0	F	Delhi	24/01/[illegible]	1[illegible]

→ replace the NAN values in name-column with

```
df1["Name"] = df1["name"].fillna("X")
df1
```

Output

	record-id	name	age	gender	city	adm-date	height-cm
0	P001	[illegible]	25.0	F	mumbai	15/01/2014	150
1	P002	Rahul	60.0	F	delhi	2014/01/16	170

(ii) Count all the null values in age column & replace them with mean of the age :-

→ `df1['age'].isna().sum()`

Output :-

24

→ `df1['age'] = df1['age'].fillna(df1['age'].mean())`
`df1.`

	patient-id	name	age	gender	city	adm-date	height-cm
0	P001	X	25.0	F	Mumbai	15/01/2014	150
1	P002	Rahul Sharma	60.0	F	Delhi	2014/01/16	170.

(iii) Find the age of person where patient-id = P014

`df1.loc[df1['Patient-id'] == "P014", ['age']]`

Output :-

	age
13	35.841057

2/02/26

Q7. Create a function called dfs & WAP of dfs in it & perform the graphs.

→
```
def dfs(st, goal, graph, vis=None):
    if vis is None:
        vis = set()
    print(st)
    if st == goal:
        return True
    vis.add(st)
    for child in graph[st]:
        if child not in vis:
            if dfs(child, goal, graph, vis):
                return True
    return False.
```

(i)
```
graph = {'A' : ['B', 'C', 'D'],
         'B' : ['A', 'C', 'E'],
         'C' : ['A', 'B', 'D', 'E'],
         'D' : ['A', 'C', 'E'],
         'E' : ['B', 'C', 'D']}
dfs('A', 'D', graph)
```

OUTPUT :-

A
B
C
D

(ii)

```
graph = {'A': ['B','C'],
         'B': ['A','D','F'],
         'C': ['A','F'],
         'D': ['B'],
         'F': ['B','C']}

dfs ('A','D', graph)
```

OUTPUT:-

A
B
D

True

iii)

```
graph = {'A': ['B','C'],
         'B': ['A','C'],
         'C': ['A','B']}

dfs ('A','C', graph)
```

OUTPUT:-

A
B
C

True

12. WAPC to create a fnc bfs & write code BFS to perform the graphs.

```
def bfs (st, goal, graph):
    vis = set()
    queue = [st]
    path = []
    vis.add(st)
    while queue:
        node = queue.pop(0)
        path.append(node)
        if node == goal:
            return path
        for neighbour in graph[node]:
            if neighbour not in vis:
                vis.add(neighbour)
                queue.append(neighbour)
    return path
```

(i)

```
graph = {'A': ['B','C','D'],
         'B': ['A','C','E'],
         'C': ['A','B','D','E'],
         'D': ['A','C','E'],
         'E': ['B','C','D']}
bfs ('A','D', graph)
```

OUTPUT:-

['A','B','C','D']

```
graph = {'A': ['B','C'],
         'B' : ['D'],
         'C' : ['E'],
         'D' : [],
         'E' : []}
bfs ('A','E', graph)
```

OUTPUT:-

['A','B','C','D','E']

(iii) =

```
graph= {'A' : ['B','C'],
        'B' : ['A','C'],
        'C' : ['A','B']}
bfs ['A','C', graph]
```

OUTPUT:-

['A','B','C']

9/02/26

Q9. perform min-max normalization on any 2 columns of a dataset with columns rollno, name, height, weight & stipen.

→

```
data set
rollno = np.arrange (1,51)
height =[]
weight =[]
stipen =[]
name= ["Riya","Rahul","Ruhi","Akanksh","Kamal"]
                                        *10.
for i in range (50):
    a = np. random. randint (150,200)
    b = np. random. randint (30,110)
    c = np. random. randint(500,1000)
    height. append (a)
    weight. append (b)
    stipen. append (c)
n = np. array((rollno, name, height, weight,
              stipen))
a = np. transpose(n)
df = pd. Data frame (a, columns=("roll no", "name",
                                 "height", "weight", "stipen"))
df
```

OUTPUT:-

	rollno	name	height	weight	stipen
0	1	Riya	179	103	824
1	2	Rahul	161	33	814
2	3	Ruhu	181	58	802

min-max normalization for height column:-

```
l = []
y = df['height'].astype(float)
min = y.min()
max = y.max()
for x in y:
    n = (x-min)/(max-min)
    l.append(n)
print(l)
```

Output:-

[0.5918367346138775, . . . , 0.0816326530612244]

min-max normalization for weight column:-

```
w = []
y = df['weight'].astype(float)
min = y.min()
max = y.max()
for x in y:
    n = (x-min)/(max-min)
    w.append(n)
print(w)
```

Output:-

[1.0, ... , 0.7866666].

THEORY:

Q1. What is graph and why do we use it?

→ Graph is just a way to show connection between things.

- points (called nodes) → the things
- lines (call edges) → the connection

why use graphs:-

→ To find shortest paths.

* google maps uses graphs to find shortest path.

→ Social Media
Who follows whom
→ Games & AI
finding path from start to goal
→ To check connectivity
Are 2 computers connected in a network.

Q2. Types of graphs
→ Connected graph:- A graph is connected if every vertex is reachable from every other vertex.
→ There is a path b/w any 2 nodes
→ No isolated part
In short → every node can reach any other node.

→ Disconnected graph → A graph is disconnected if some vertices cannot be reached from other
It has two/more separate parts.
in short words: some ~~to~~ nodes cannot reach each other.

→ Cyclic graph:- If it contains at least 1 cycle. A cycle means you can start at a node & come back to it by following edges.

Q3 What is BFS? How do we use it & real life application of BFS & how?

→ BFS is a graph traversal algorithm.
BFS explores a graph level by level, starting from a given node, visible all neighbour first, then moving to the neighbour of neighbour.

Real life Application

1) Google Maps / Shortest path.
→ BFS helps find the shortest path between 2 locations when all roads have equal weight.

2) Social Networks
used in platform like instagram
Ex:- Finding mutual friends or connection

3) Web Crawlers
Search engines like Google use BFS to crawl websites.

4) Network Broadcasting
In computer networks, BFS is used to send messages across all nodes efficiently
Server → Router → Multiple devices.

5) AI & Games

⇒ BFS is used in game pathfinding

Example :-

- Chess move search
- Robot navigation
- Maze solving.

→ BFS is used because

* Finds shortest path in unweighted graphs.
* Explores nodes systematically level by level.
* Easy to implement with queue.

Ques 4: What is dfs and why do we use it? Real life applications.

⇒ DFS is a fundamental graph traversal ~~graph traversal~~ algorithm that explores as far as possible along each branch before backtracking.

Its real life application include solving mazes, web crawling, network routing

1) Topological sorting
2) Pathfinding & Maze solving
3) Cycle detection
4) Finding strongly connected components
5) AI & game theory

Ques :- Design and develop a program to implement the dfs algorithm for graph traversal in artificial intelligence in the following specification

16-01-26

(i) Construct a static graph consisting of 5 nodes & 10 edges where the graph is stored using a dictionary representation, the graph is given as : a-b, a-c, b-d, b-e, c-d, c-f, d-e, d-f, e-f, b-f

(ii) Create the graph where the no of edges are provided by the user at run time.

(iii) Accept the initial and the goal node from the user.

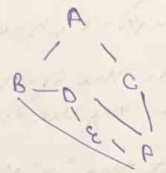
(iv) Display the path from the initial node to the goal node for both cases

⇒ (i)

```
def dfs (st, goal, graph, vis=None):
    if vis is None:
        vis = set ()
    print (st)
    if st == goal:
        return True
    vis.add (st)
    for child in graph[st]:
        if child not in vis:
            if dfs (child, goal, graph,
                    vis):
                return ~~fal~~ True
    return False
```

```
graph = {'a': ['b', 'c'], 'b': ['d', 'e', 'f'], 'c':
  ['d', 'f'], 'd': ['e', 'f'], 'e': ['f'], 'f': []}
a = dfs ('a', 'c', graph)
a.
```

→ a
b
d
e
f
c
True



Ques2: Design and develop a program to implement bfs algorithm for graph traversal in as with the following specifications

(i) Construct a static graph consisting of 6 nodes and 10 edges where the graph is stored using a dictionary representation. The graph is given a-b, a-c, b-d, b-e, c-d, c-f, d-e, d-f, e-f, b-f

(ii) Create another graph dynamically where the no of nodes & edges are given by the user at runtime and the user specifies which nodes are connected

(iii) Accept the initial node, goal node from the user

(iv) Perform the bfs from the given starting node to goal node.

(v) Display the order of node traversal

(vi) Display the shortest path for both static & dynamic case

(ii) and (iii)

```
(ii) graph = {}
    n = int (input ("enter no of nodes"))
    for i in range (n):
        node = input ("enter node name")
            graph [node] = [ ]
    e = int (input ("enter number of edges"))
    for i in range (e):
        u = input ("enter starting node:")
        v = input ("enter ending node:")
        graph [u]. append (v)
        graph [v] . append (u)
    for key in graph:
        print (key, ':', graph [key])
```

Output

```
enter no of nodes 6
enter node name a
  enter node name b
  "      "     "   c
    "    "     "   d
    "    "     "   e
    "    "     "   f
enter no of edges 10
enter starting node a
enter ending node b
  "     starting "  a
    "   ending   "  c
```

enter starting node b
" ending " d
" starting " b
" ending " e
" starting " c
" ending node d
" starting " e
" ending " f
" starting " d
" ending " e
" starting node d
" ending " f
" starting " e
" ending " f
" starting " b
" ending " f.

a : ['b', 'c']
b : ['a', 'd', 'e', 'f']
c : ['a', 'd', 'f']
d : ['b', 'c', 'e', 'f']
e : ['b', 'd', 'f']
f : ['c', 'd', 'e', 'b']

CODE :- x = dfs ('a', 'c', graph)
x

OUTPUT:- a
b
d
e
f
True

iii) initial = input("enter start node")
final = input("enter final node")
x = dfs (initial, final, graph)
x

Output:-
enter start node a
enter final node c

a
b
d
e
f
c
True

Answer 2 (i) static case

```
def bfs (st, goal, graph):
    vis = set()
    queue = [st]
    path = []
    vis.add(st)
    while queue:
        node = queue.pop(0)
        path.append(node)
        if node == goal:
            return path
        for neighbor not in vis:
            vis.add(neighbor)
            queue.append(neighbor)
return path
graph = {'a':['b','c'],'b':['d','e','f'],'c':['d','f'],
'd':['e','f'],'e':['f'],'f':[]}
x = bfs ('a','c',graph)
x
```

OUTPUT:-

['a', 'b', 'c']

(ii) Dynamic

```
graph = {}
n = int(input ("enter no of nodes"))
for i in range (n):
        node = input ("enter node name")
        graph = [node] = []
e = int(input ("enter no of edges"))
for i in range (e):
        u = input ("enter starting node")
        v = input ("enter ending node")
        graph[u]. append (v)
        graph1 [v]. append (u)
for key in graph1
        print (key, ':', graph1 [key])
```

Output

```
enter no of nodes 6
enter node name a
"      "    "   b
                c
                d
                e
                f
enter no of edges 10
enter starting node a
        ending   b
        starting a
        ending   c
        starting b
        ending   d
enter starting node b
"      ending   e
"      starting c
"      ending   d
"      starting c
"      ending   f
"      starting d
"      ending   e
"      starting d
"      ending   f
"      starting b
       ending   f
```

a : ['b', 'c']

b : ['a', 'd', 'e', 'f']

c : ['a', 'd', 'f']

d : ['b', 'c', 'e', 'f']

e : ['b', 'd', 'f']

f : ['c', 'd', 'e', 'b']

CODE :-

```
u1 = bfs ('a', 'c', graph1)
u1
```

OUTPUT ['a', 'b', 'c']

(iii)

```
initial = input ("enter start node")
final = input ("enter final node")
x2 = bfs (initial, final, graph1)
x2
```

OUTPUT

→ ['a', 'b', 'c']

```
xs = bfs(initial, final, graph)
xs
```

OUTPUT

['a', 'b', 'c']

Ques: Code for applying standard scaler to a list named data.

→

```
def standard(data):
    mean = sum(data)/len(data)
    sumsq = 0
    for x in data:
        sumsq += (x-mean)**2
    std = (sumsq/len(data)) ** 0.5
    scaled = []
    for x in data:
        scaled.append((x-mean)/std)
    return scaled
data = [10,20,30,40,50]
print(standard(data))
```

OUTPUT

[-1.414, -0.707, 0.0, 0.707, 1.414]

Ques: Write a code to implement a star to a graph with given heuristic values as:-

A=6, B=4, C=4, D=2, E=1, F=0

Graph

→

```
def astar(start, goal, graph, h):
    open = [start]
    g = {start: 0}
    parent = {start: None}
    while open:
        n = open[0]
        for i in open:
            if g[i]+h[i] < g[n]+h[n]:
                n = i
        if n == goal:
            path = []
            while n:
                path.append(n)
                n = parent[n]
            return path[::-1]
        open.remove(n)
        for m in graph[n]:
            if m not in g:
                g[m] = g[n] + graph[n][m]
                parent[m] = n
                open.append(m)
```

```
graph = {'A':{'B':2, 'E':3},
'B':{'D':3, 'G':2}, 'C':{'F':5}, 'D':{},
'E':{'F':2}, 'F':{}}
h = {'A':6, 'B':4, 'C':4, 'D':2, 'E':2, 'F':0}
print(astar('A', 'F', graph, h))
```

Output

['A', 'B', 'E', 'F']

Ques IQR method code to identify outliers in a given series of data.

→
```
import pandas as pd
data = pd.series([10, 12, 14, 15, 18, 20, 100])
q1 = data.quantile(0.25)
q3 = data.quantile(0.75)
iqr = q3 - q1
lower = q1 - 1.5 * iqr
upper = q3 + 1.5 * iqr
outliers = data[(data < lower) | (data > upper)]
print(f" Outliers : {outliers.tolist()}")
```

Output

Outliers : [100.0]

(16-03-25)

Ques. Applying linear regression model on a dataset & calculating MSE, MAE, RMSE, r^2, score and plotting the graph of the best fit line.

→ Step 1. import numpy as np
- import pandas as pd
- from sklearn.linear_model import LinearRegression
- from sklearn.model_selection import train_test_split
- from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score.
- import matplotlib.pyplot as plt

Step 2:- data = {"x": [1,2,3,4,5,6,7,8], "y": [2,4,6,8, 10,12,14,16]}
df = pd.DataFrame(data)
df

OUTPUT

	x	y
0	1	2
1	2	4
2	3	6
3	4	8
4	5	10
5	6	12
6	7	14
7	8	16

Step 3:- X = df[['x']]
Y = df['y']

Step 4:- X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=42)

Step 5:- X_train, X_test, Y_train, Y_test

Step 6:- model = linear Regression()

Step 7:- model.fit (x_train, y_train

Output

▼ LinearRegression
▶ Parameters

Step 8:- y_pred = model.predict (x_test)

Step 9:- y_pred out: array([4, 12])

Step 10:- y_test

Out

```
1    4
5   12
```

Name: y, dtype: int64

* Output of step 5

```
(
0    1
1    8
2    3
4    5
3    4
6    7 ,
     x
1    2
5    6 ,
0    2
1   16
2    6
4   10
3    8
6   14
```

Name: y, dtype: int64

```
1    4
5   12
```

Name: y, dtype: int64

Step 11:- slope = model.coef_[0]
intercept = model.intercept_
slope, intercept

Out: (1.9999999..... (15), 5.329070518200751e-15)

Step 12:- mean_absolute_error(y_test, y_pred)

Out: 2.6645352591003757e-15

Step 13:- mean_squared_error(y_test, y_pred)

Out: 7.888609052210118e-30

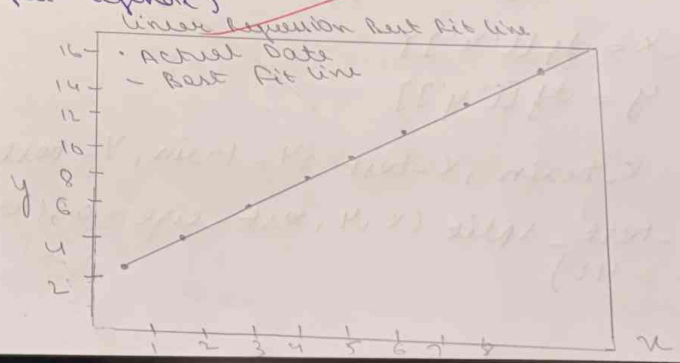
Step 14:- np.sqrt (mean_squared_error(y_test, y_pred

Out: 2.808666774861361e-15

Step 15:- r2_score (y_test, y_pred)

Out: 1.0

Step 16:- plt.scatter(x, y, color="blue", label="Actual Data")
plt.plot (x, model.predict(x), color="red", label="Best fit line")
plt.xlabel ("x")
plt.ylabel ("y")
plt.title ("Linear Regression Best fit line")
plt.legend()



Ques2: Applying linear Regression model on a dataset and calculating MSE, RMSE, MAE, r2, RA2 & Plotting the graph of best fit line.

⇒ Step 1 : import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn-model import linear Regression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

Step 2 data = {"x": [11,12,13,14,15,16,17,18],
"y": [22,23,33,11,12,14,16,17]}

df = pd.DataFrame(data)

df

Out

	x	y
0	11	22
1	12	23
2	13	33
3	14	11
4	15	12
5	16	14
6	17	16
7	18	17

Step 3 - x = df[['x']]
y = df['y']

Step 4 - X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size = 0.2, random_state = 42)

Step 5 - X_train, X_test, Y_train, Y_test

Out

```
   x
0  11
7  18
2  13
4  15
3  14
6  17,
   x
1  12
5  16,
0    22
7    17
2    33
4    12
3    11
6    16
Name: y, dtype: int64,
1  23
5  14
```

Step 6 - model = linear Regression()

Step 7 - model.fit(X_train, Y_train)

Out

▽ LinearRegression
▷ Parameters

Step 8 - Y_pred = model.predict(X_test)

Step 9 - Y_pred

Out array([22.1, 16.7])

Step 10 Y_test

Out

```
1  23
5  14
Name: y, dtype: int64
```

Step 11 slope = model.coef_[0]
intercept = model.intercept_
slope, intercept

Out: (-1.3499999999999999, 38.3)

Step 12 mean_absolute_error (Y_test, Y_pr

Out: 1.8000000007

Step 13: mean_squared error (Y_test, Y_pr

Out: 4.05

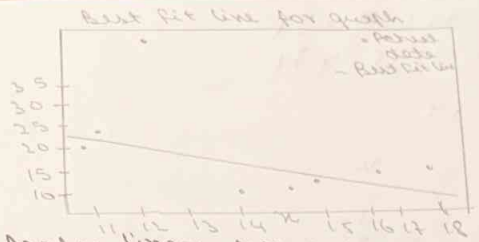
Step 14: np.sqrt (mean_squared_error (Y_test, Y_pred))

Out: 2.0124611797498106

Step 15: r2_score (Y_test, Y_pred)

Out: 0.8

Step 16 plt.scatter (x, y, colour="blue", label="Actual Data") plt.plot (x, model.predict(x), colour="red", label="Best fit line")
plt.xlabel("x")
plt.ylabel("y")
plt.title ("Best fit line for graph")
plt.legend()



Q3. Apply linear regression model on dataset & calculate accuracy_score, precision_score, recall_score, f1_score, confusion_matrix

→ Step 1. import numpy as np
import pandas as pd
· from sklearn.neighbors import KNeighborsClassification.
· from sklearn.model_selection import train_test_split
· from sklearn.metrics import accuracy_score, precision_score, recall_score, confusion_matrix, classification_report, f1_score

Step 2 data = {"x": [1,2,3,4,5], "y": [0,0,1,1,1]}
df = pd.DataFrame(data)
df

Out

	x	y
0	1	0
1	2	0
2	3	1
3	4	1
4	5	1

Step3 x = df[['x']]
y = df[['y']]

Step4 x, y

Out (

	x
0	1
1	2
2	3
3	4
4	5 ,

	y
0	0
1	0
2	1
3	1
4	1)

Step5 x_train, x_test, y_train, y_test = train_test_split (x, y, test_size = 0.2, random_state = 42)

Step6 KNN = KNeighborsClassification(n_neighbors = 3)

Step7 knn.fit (x_train, y_train)

▼ KNeighborsClassifier
▷ Parameters

Step8 y_pred = knn.predict (x_test)

Step9 y_test Out: y
1 0

Step10 y_predict
Out: array([1])

Step11 accuracy_score (y_test, y_pred)
Out: 0.0

Step12 precision_score (y_test, y_pred)
Out: 0.0

Step13 recall_score (y_test, y_pred)
Out: 0.0

Step14 print(confusion_matrix (y_test, y_pred
Out: [[0 1]
[0 0]]

Step15 print (classification_report (y_test, y_pred))

Abhi 23-01-26

Out:

	precision	recall	f1-score	support
0	0.00	0.00	0.00	1.0
1	0.00	0.00	0.00	0.0
accuracy				
macro avg	0.00	0.00	0.00	1.0
weighted avg	0.00	0.00	0.00	1.
				1.